

Name: Tanmay Kathar

Team: Cloud

Date: 11/07/2024

Task: Provision AWS Resources for Kubernetes Cluster with Terraform.

Solution:

Step 1: Install Terraform on Windows

1) Download Terraform:

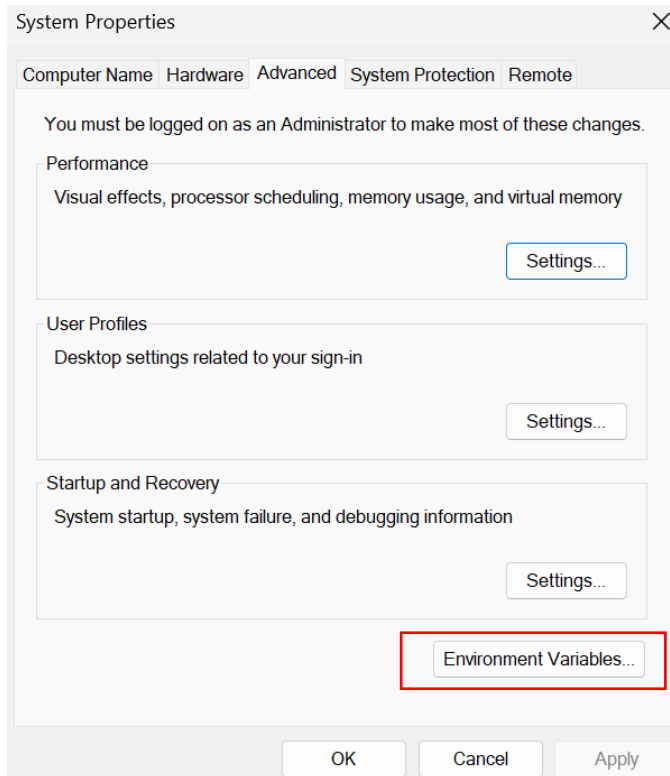
- Go to the [Terraform downloads page](#).
- Download the appropriate version for Windows.

2) Install Terraform:

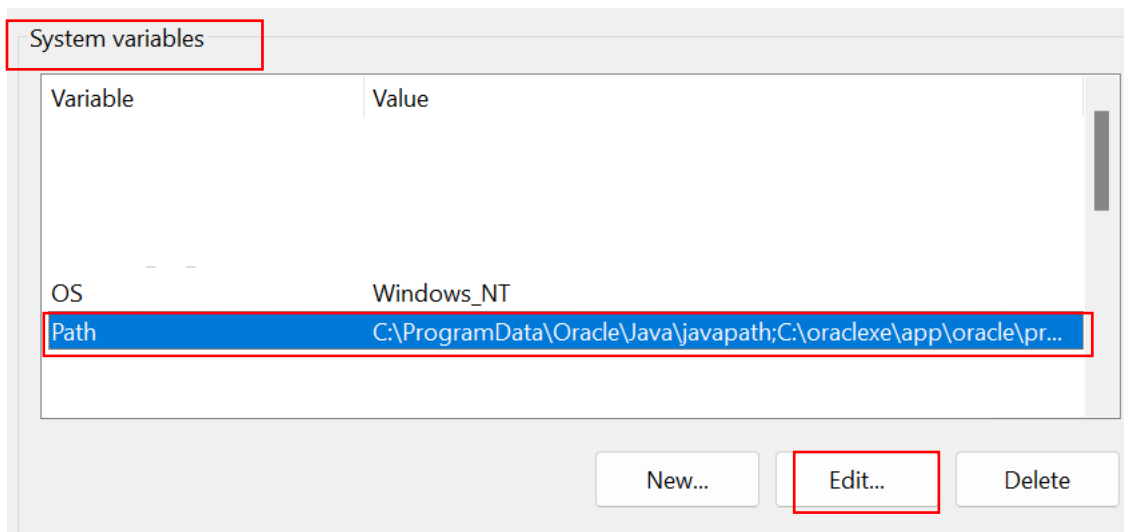
- Extract the downloaded zip file.
- Move the terraform.exe file to a directory included in your system's PATH (e.g., C:\Program Files\Terraform).
- Copy (e.g., C:\Program Files\Terraform) Path to your system “Environment variable”.

Set terraform path in Environment variable using below images.

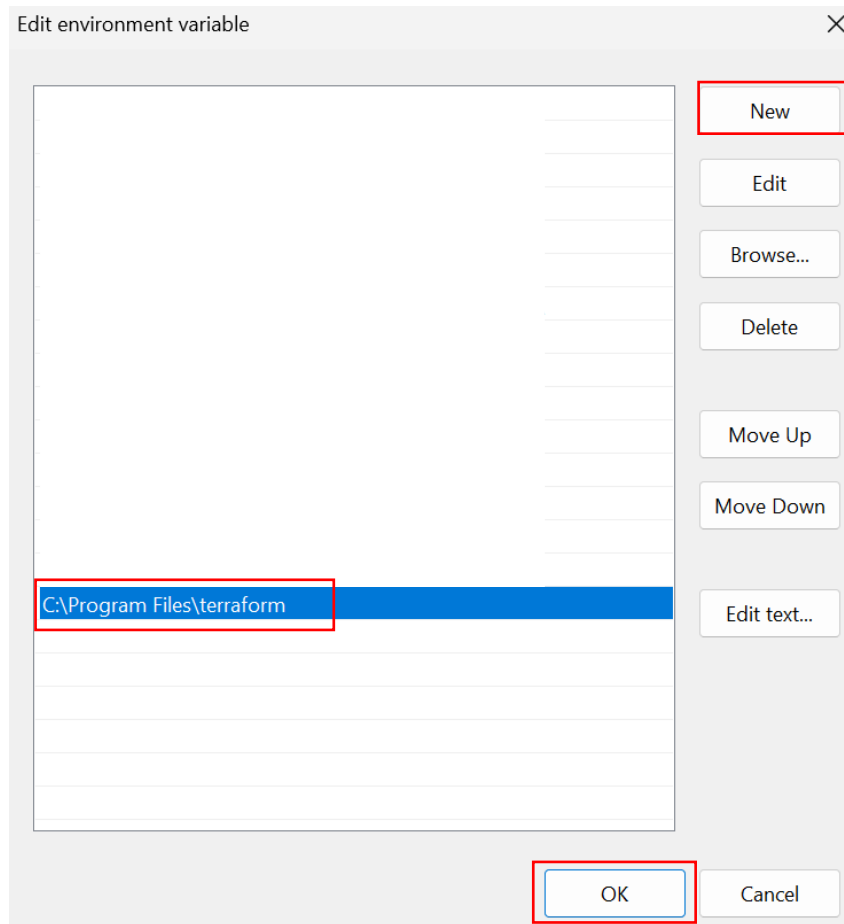
- Search Environment variable in windows search bar or else in settings.
- Click on Environment variable.



- In system variable Click on “Path” and then click “Edit”.



- Add new path



Click “ok” and environment variable path is set for terraform.

3) To check Terraform is install in the system:

- Search cmd and Run as administrator
- cd C:\Program Files\Terraform
- terraform --version

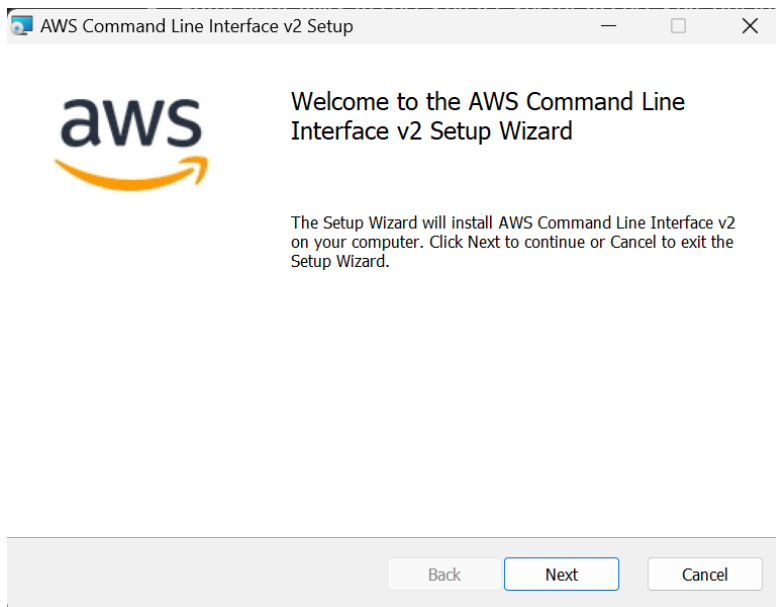
```
C:\Windows\System32>cd C:\Program Files\Terraform
C:\Program Files\terraform>terraform --version
Terraform v1.9.0
on windows_amd64
```

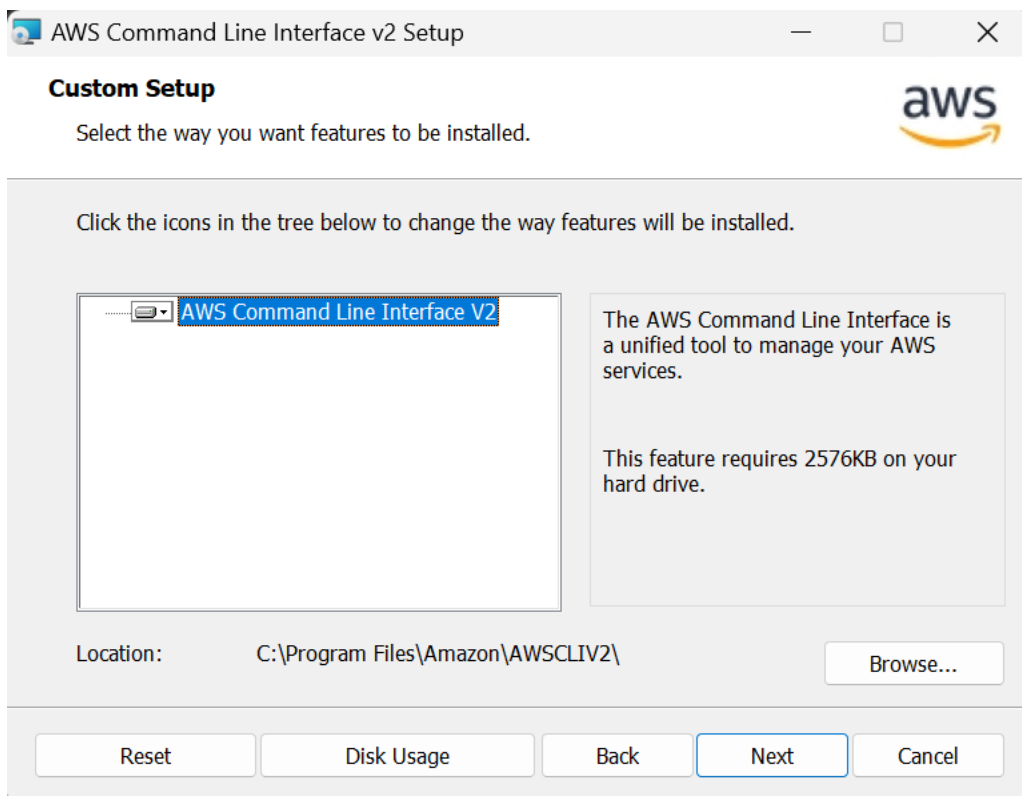
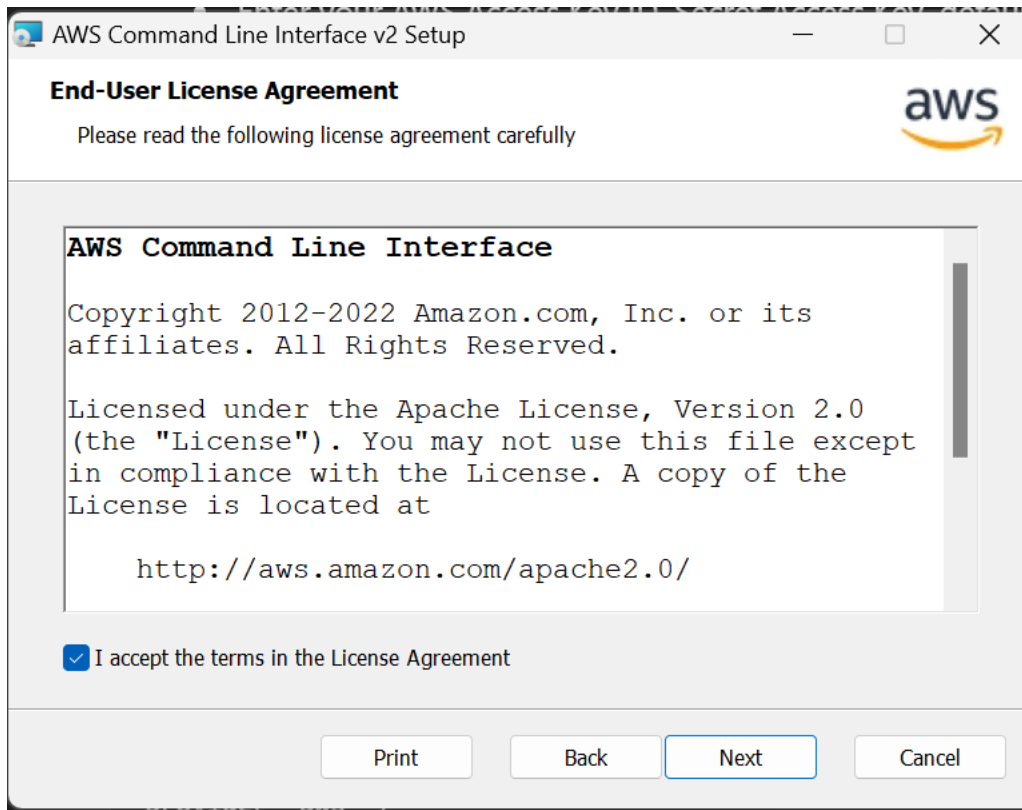
Step 2: Configure AWS Credentials for Terraform.

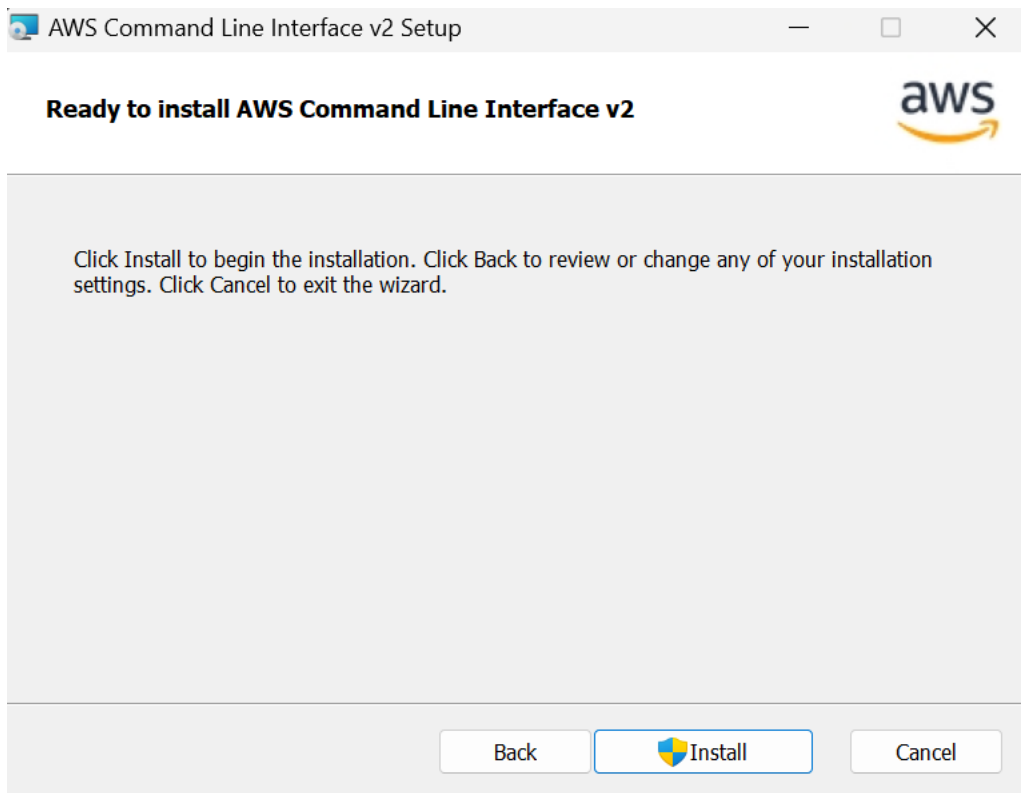
1) Install AWS CLI (if not already installed):

- Download the AWS CLI from the [AWS CLI installation page](https://awscli.amazonaws.com/AWSCLIV2.msi).
- Follow the installation instructions.

```
C:\Program Files>cd..  
C:\>msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```







2) Create IAM user to get AWS Access Key ID, Secret Access Key.

- Go to IAM in AWS console.
- Create user by following below images.

User details

User name

tanmay

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☐ Provide user access to the AWS Management Console - *optional*
If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

i If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

Cancel Next

- Select **“Attach policies directly”**
- Select **“Administrator Access”**

Permissions options

☐ Add user to group
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1/1221)

Choose one or more policies to attach to your new user.

Filter by Type

272 matches

	Policy name	Type	Attached entities
☒	AdministratorAccess	AWS managed - job function	0

Review and create the user.

- Click on **Created user**
- At top in Summary click **“create access key”** and follow below step.

IAM > Users > terraform-user > Create access key

Step 1

Access key best practices & alternatives

Step 2 - optional

Set description tag

Step 3

Retrieve access keys

Access key best practices & alternatives

Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.

Use case

☒ **Command Line Interface (CLI)**
You plan to use this access key to enable the AWS CLI to access your AWS account.

☐ Local code
You plan to use this access key to enable application code in a local development environment to access your AWS account.

Set description tag - *optional* [Info](#)

The description for this access key will be attached to this user as a tag and shown alongside the access key.

Description tag value

Describe the purpose of this access key and where it will be used. A good description will help you rotate this access key confidently later.

Maximum 256 characters. Allowed characters are letters, numbers, spaces representable in UTF-8, and: _ . : / = + - @

Cancel

Previous

Create access key

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

Download .csv file

Done

Download .csv file for further steps.

Create password for IAM user

- Click on created user.
- Go to security credentials and set user password.

Enable console access

Enable console access for terraform-user.

Console password

☐ Autogenerated password

☒ Custom password

.....

- Must be at least 8 characters long
- Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # \$ % ^ & * () _ + - (hyphen) = [] { } | ' "

☐ Show password

☐ User must create new password at next sign-in
Users automatically get the [IAMUserChangePassword](#) policy to allow them to change their own password.

Cancel

Enable console access

Console password

You have successfully enabled the user's new password.

This is the only time you can view this password. After you close this window, if the password is lost, you must create a new one.

Console sign-in URL

<https://637423437444.signin.aws.amazon.com/console>

User name

tanmay

Console password

***** [Show](#)

Download .csv file

Close

Download the .csv file to take user access.

3) Configure AWS CLI:

- Open Command Prompt (cmd) and run “**aws configure**”.
- Enter your AWS Access Key ID, Secret Access Key, default region name (e.g., us-east-2), and default output format (e.g., json).

```
C:\>aws configure
AWS Access Key ID [*****Z7ZN]
AWS Secret Access Key [*****z3/Q]:
Default region name [us-east-2]:
Default output format [json]:
```

For default region name (choose your region)

For default output format Press “enter”.

Step 3: Create a Terraform Configuration for the EC-2 and EBS volume.

1) Create a Directory for Terraform Files:

- Create a new directory for your Terraform project, e.g., C:\terraform\kubernetes.

2) Create main.tf File:

- Inside the project directory, create a file named main.tf and add the following Terraform configuration:

```
provider "aws" {  
  region = "us-east-2" # Specify your AWS region  
}  
  
# Variables  
  
variable "key_name" {  
  description = "Name of the SSH key pair to use for EC2 instances"  
  default = "mykey" # Replace with your SSH key pair name  
}  
  
variable "instance_type_control_plane" {  
  description = "Instance type for Kubernetes control plane node"  
  default = "m5.large" # Adjust based on Kubernetes recommendations  
}  
  
variable "instance_type_worker_node" {  
  description = "Instance type for Kubernetes worker node"  
  default = "m5.large" # Adjust based on Kubernetes recommendations  
}  
  
variable "ami" {  
  description = "AMI ID for Ubuntu 20.04 LTS"  
  default = "ami-094be4c7f1e506a7a" # Ubuntu 20.04 LTS AMI ID in us-east-2  
}  
  
# Security Group  
  
resource "aws_security_group" "kubernetes_sg" {  
  name = "kubernetes-sg"  
  description = "Security group for Kubernetes cluster"  
  vpc_id = "vpc-04c1dab63e3a8b6c8" # Replace with your VPC ID
```

```
// Inbound rules (as previously defined)
```

```
ingress {
```

```
  description = "SSH access"
```

```
  from_port = 22
```

```
  to_port = 22
```

```
  protocol = "tcp"
```

```
  cidr_blocks = ["0.0.0.0/0"]
```

```
}
```

```
ingress {
```

```
  description = "Kubernetes API Server"
```

```
  from_port = 6443
```

```
  to_port = 6443
```

```
  protocol = "tcp"
```

```
  cidr_blocks = ["0.0.0.0/0"]
```

```
}
```

```
ingress {
```

```
  description = "etcd server client API"
```

```
  from_port = 2379
```

```
  to_port = 2380
```

```
  protocol = "tcp"
```

```
  cidr_blocks = ["0.0.0.0/0"]
```

```
}
```

```
ingress {
```

```
  description = "Kubelet API"
```

```
  from_port = 10250
```

```
  to_port = 10250
```

```
  protocol = "tcp"
```

```
  cidr_blocks = ["0.0.0.0/0"]
```

```
}
```

```
ingress {  
  description = "kube-scheduler"  
  from_port = 10251  
  to_port = 10251  
  protocol = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}  
  
ingress {  
  description = "kube-controller-manager"  
  from_port = 10252  
  to_port = 10252  
  protocol = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}  
  
ingress {  
  description = "NodePort Services"  
  from_port = 30000  
  to_port = 32767  
  protocol = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}  
}
```

```
# Resources
```

```
resource "aws_instance" "control_plane" {
```

```
ami = var.ami
```

```
instance_type = var.instance_type_control_plane
```

```
key_name = var.key_name
```

```
security_groups = [aws_security_group.kubernetes_sg.id]
```

```
count = 1 # One control plane instance
```

```
subnet_id = "subnet-0c92ebb227d47df64" # Replace with your subnet ID
```

```
tags = {
```

```
  Name = "k8s-control-plane"
```

```
}
```

```
}
```

```
resource "aws_instance" "worker_node" {
```

```
ami = var.ami
```

```
instance_type = var.instance_type_worker_node
```

```
key_name = var.key_name
```

```
security_groups = [aws_security_group.kubernetes_sg.id]
```

```
count = 1 # One worker node instance
```

```
subnet_id = "subnet-0c92ebb227d47df64" # Replace with your subnet ID
```

```
tags = {
```

```
  Name = "k8s-worker-node"
```

```
}
```

```
}
```

3) Initialize Terraform:

- Open Command Prompt and navigate to your project directory.
- Run “**terraform init**” to initialize the project.

```
C:\Program Files\terraform\kubernetes>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.57.0...
- Installed hashicorp/aws v5.57.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

4) Apply Terraform Configuration:

- Run “**terraform apply**”.
- Review the plan output and confirm the apply by typing “yes”.

```
C:\Program Files\terraform\kubernetes>terraform apply
aws_security_group.kubernetes_sg: Refreshing state... [id=sg-048d808223567ac41]

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply" which may have
affected this plan:

# aws_security_group.kubernetes_sg has been deleted
- resource "aws_security_group" "kubernetes_sg" {
  - id           = "sg-048d808223567ac41" -> null
    name        = "kubernetes-sg"
  # (9 unchanged attributes hidden)
}

Unless you have made equivalent changes to your configuration, or ignored the relevant attributes using ignore_changes,
the following plan may include actions to undo or respond to these changes.

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
```

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

aws_security_group.kubernetes_sg: Creating...

aws_security_group.kubernetes_sg: Creation complete after 6s [id=sg-0613e97b535860c42]

aws_instance.control_plane[0]: Creating...

aws_instance.worker_node[0]: Creating...

aws_instance.worker_node[0]: Still creating... [10s elapsed]

aws_instance.control_plane[0]: Still creating... [10s elapsed]

aws_instance.control_plane[0]: Creation complete after 18s [id=i-0d87351e8320dd560]

aws_instance.worker_node[0]: Creation complete after 18s [id=i-010a0462505236a1e]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Step 4: Verify the EC2 Creation in AWS Console.

- Log in to your AWS Management Console.
- Navigate to the EC-2 dashboard.
- Verify that a new “EC-2 of control plane and worker node” is created in us-east-2 region.

Instances (2) Info

Refresh

Connect

Instance state ▾

Actions ▾

Launch instances

Find Instance by attribute or tag (case-sensitive)

All states ▾

running X

Clear filters

< 1 >

⚙️

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status
☐	k8s-control-plane	i-0d87351e8320dd560	Running	m5.large	2/2 checks passed	View alarms
☐	k8s-worker-node	i-010a0462505236a1e	Running	m5.large	2/2 checks passed	View alarms

➤ Security group for kubernetes provision

☐	Name	Security group ID	Security group name	VPC ID
☐	-	sg-0613e97b535860c42	kubernetes-sg	vpc-04c1dab63e

➤ Inbound Rules

Inbound rules (7)

Manage tags

Edit inbound rules

Q Search

< 1 > ⚙

☐	Name	Security group rule...	IP version	Type	Protocol	Port range
☐	-	sgr-0aa0de4e35e9e5c74	IPv4	Custom TCP	TCP	10252
☐	-	sgr-0cafd23c0b5a66ebf	IPv4	Custom TCP	TCP	10251
☐	-	sgr-00b4292809c3e9d...	IPv4	Custom TCP	TCP	6443
☐	-	sgr-0b25e4a21f070b74f	IPv4	Custom TCP	TCP	10250
☐	-	sgr-0ea1c375f22b1e811	IPv4	Custom TCP	TCP	2379 - 2380
☐	-	sgr-000f5f062c78f8225	IPv4	Custom TCP	TCP	30000 - 32767
☐	-	sgr-0e97c5a75f2796976	IPv4	SSH	TCP	22